

```
import kagglehub
```

```
path = kagglehub.dataset_download("kmader/skin-cancer-mnist-ham10000")
```

Using Colab cache for faster access to the 'skin-cancer-mnist-ham10000' dataset.

```
import os
```

```
import pandas as pd
```

```
metadata_path = os.path.join(path, "HAM10000_metadata.csv")
```

```
df = pd.read_csv(metadata_path)
```

```
img_dir1 = os.path.join(path, "HAM10000_images_part_1")
```

```
img_dir2 = os.path.join(path, "HAM10000_images_part_2")
```

```
def get_path(img_id):
```

```
    p1 = os.path.join(img_dir1, img_id + ".jpg")
```

```
    p2 = os.path.join(img_dir2, img_id + ".jpg")
```

```
    return p1 if os.path.exists(p1) else p2
```

```
df['path'] = df['image_id'].apply(get_path)
```

```
malignant = ['mel', 'bcc', 'akiec']
```

```
df['label'] = df['dx'].apply(lambda x: 1 if x in malignant else 0)
```

```
from torchvision import transforms
```

```
train_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2),
    transforms.ToTensor()
])
```

```
val_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor()
])
```

```
from torch.utils.data import Dataset
```

```
from PIL import Image
```

```

class SkinDataset(Dataset):
    def __init__(self, df, transform):
        self.df = df.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img = Image.open(self.df.loc[idx, 'path']).convert("RGB")
        label = self.df.loc[idx, 'label']

        return self.transform(img), label

```

```

from sklearn.model_selection import train_test_split

train_df, val_df = train_test_split(
    df,
    test_size=0.2,
    stratify=df['label'],
    random_state=42
)

```

```

from torch.utils.data import DataLoader

train_loader = DataLoader(
    SkinDataset(train_df, train_transform),
    batch_size=32,
    shuffle=True
)

val_loader = DataLoader(
    SkinDataset(val_df, val_transform),
    batch_size=32,
    shuffle=False
)

```

```

import torch
import torch.nn as nn
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

device = torch.device("cpu")

class SkinModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT)

        in_features = self.model.classifier[1].in_features

```

```

        self.model.classifier = nn.Sequential(
            nn.Linear(in_features, 256),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(256, 1)
        )

    def forward(self, x):
        return self.model(x)

model = SkinModel().to(device)

```

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f58
 100%|██████████| 20.5M/20.5M [00:00<00:00, 139MB/s]

```

import torch
print(torch.cuda.is_available())

```

True

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)

```

cuda

```

model = model.to(device)

```

```

for imgs, labels in train_loader:
    imgs = imgs.to(device)
    labels = labels.float().to(device)

```

```

train_loader = DataLoader(
    SkinDataset(train_df, train_transform),
    batch_size=64,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

```

```

scaler = torch.cuda.amp.GradScaler()

```

```

/tmp/ipykernel_14359/2340218076.py:1: FutureWarning: `torch.cuda.amp.GradScaler(
  scaler = torch.cuda.amp.GradScaler()

```

```
import torch
import torch.nn as nn

pos_weight = torch.tensor([
    (len(train_df) - train_df['label'].sum()) / train_df['label'].sum()
]).to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

```
for epoch in range(30):

    model.train()

    for imgs, labels in train_loader:
        imgs = imgs.to(device)
        labels = labels.float().to(device)

        with torch.cuda.amp.autocast():
            outputs = model(imgs).squeeze()
            loss = criterion(outputs, labels)

        optimizer.zero_grad()
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

    print(f"Epoch {epoch+1} completed")
```

```
/tmp/ipykernel_14359/1909661804.py:9: FutureWarning: `torch.cuda.amp.autocast(ar
with torch.cuda.amp.autocast():
```

```
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Epoch 6 completed
Epoch 7 completed
Epoch 8 completed
Epoch 9 completed
Epoch 10 completed
Epoch 11 completed
Epoch 12 completed
Epoch 13 completed
Epoch 14 completed
Epoch 15 completed
Epoch 16 completed
Epoch 17 completed
Epoch 18 completed
Epoch 19 completed
Epoch 20 completed
```

```
Epoch 21 completed  
Epoch 22 completed  
Epoch 23 completed  
Epoch 24 completed  
Epoch 25 completed  
Epoch 26 completed  
Epoch 27 completed  
Epoch 28 completed  
Epoch 29 completed  
Epoch 30 completed
```

```
import numpy as np  
  
model.eval()  
  
probs = []  
y_val = []  
  
with torch.no_grad():  
    for imgs, labels in val_loader:  
        imgs = imgs.to(device)  
  
        outputs = torch.sigmoid(model(imgs)).cpu().numpy()  
  
        probs.extend(outputs)  
        y_val.extend(labels.numpy())  
  
probs = np.array(probs).flatten()  
y_val = np.array(y_val)
```

```
import numpy as np  
  
probs = np.array(probs)
```

```
import numpy as np  
  
probs = np.array(probs)  
  
threshold = 0.35  
y_pred = (probs > threshold).astype(int)
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score  
  
accuracy = accuracy_score(y_val, y_pred)  
precision = precision_score(y_val, y_pred)  
recall = recall_score(y_val, y_pred)  
  
print(f"Accuracy : {accuracy:.4f}")  
print(f"Precision: {precision:.4f}")
```

```
print(f"Recall    : {recall:.4f}")
```

```
Accuracy : 0.9141
```

```
Precision: 0.7483
```

```
Recall    : 0.8440
```